



Presenter 3 — Heddle, Weave & live demo (part A)

Slides 12 + live scenarios 1, 2, 3, 4, 7 · target time ≈ 9 minutes (4 min slide, 5 min demo)

Your name: _____ Roll no.: _____

You take over from: Presenter 2 (cryptography & Warp) · You hand over to: Presenter 4 (demo part B)

How to use this script

The **Say** text is written to be spoken at a natural pace (about 130 words a minute). You do not need to read it word for word: learn the first and last sentence of each slide and the numbers, and speak the middle in your own words. *Italic cues* tell you what to point at. Everything technical here matches the code in the repository.

The whole talk at a glance

Part	Presenter	Slides	Time	What it covers
1	Presenter 1	1 – 7	≈ 8 min	Title, abstract, introduction, problem & objectives, literature review (2 slides), research gap
2	Presenter 2	8 – 11	≈ 8 min	System architecture, install flow, attestation & logging flow, cryptography & the Warp log
3	Presenter 3	12 + live demo A	≈ 9 min	Heddle sandbox & Weave policy engine; live scenarios 1, 2, 3, 4, 7
4	Presenter 4	live demo B + 13 – 16	≈ 9 min	Live scenarios 5, 6, 8 and the full-run console; results, demo recap, conclusion, references, Q&A;

Slide 12 · Heddle sandbox and Weave policy engine (3 min 30 s)

On screen: left half first, from the outer box inwards; then the red box; then the nine rule chips and the three-node graph.

Say

On the left is **Heddle**, our build sandbox, made of three nested layers. The outer layer is Linux **namespaces**: the build gets its own user, mount, network, process, IPC, hostname and cgroup namespaces. We `pivot_root` into a fresh, empty temporary filesystem that contains only the toolchain, read-only, and the build directory. Your home directory does not exist inside, and the network namespace is empty.

The middle layer is **Landlock**, a Linux security module that any unprivileged process can apply to itself. It is a kernel allow-list for files, TCP ports and inter-process communication. The inner layer is a **seccomp-BPF** filter that refuses dangerous system calls such as `mount`, `ptrace`, `bpf` and `io_uring`, and hands socket and file-open calls to a supervisor that **explains** each denial in the terms of our requirements.

Tiers: if the host disallows user namespaces we drop to the reduced tier, Landlock plus seccomp, and say so. If even Landlock is missing, we **refuse to build**. There is never an unconfined fallback.



The red box is a change we made while hardening the prototype. A malicious script can wrap its payload in `|| true` so that the build still succeeds after the sandbox denies it. We now treat any attempt to read credentials, or to use the network, as hostile: rebuilders refuse to attest such a build, and Weave blocks it, even when it exits with success.

On the right is **Weave**. It is a pure function from the policy and the evidence to a decision. It evaluates all nine rules and reports every violation, each with evidence and a remediation. Continuity runs first, because a change of maintainer can tighten the rules that follow.

The most interesting algorithm is **independence**. Two attestations are correlated if they share a rebuilder, an organisation or a toolchain; the toolchain ID is a SHA-256 hash of the build image and its component versions. The number of independent attestations is the **maximum independent set** of that correlation graph. In this example, thread-b and thread-c used the same toolchain, so three agreeing attestations count as two. That still meets the default k of two.

Terms to be ready to explain: namespaces, pivot_root, Landlock ABI, seccomp-BPF, user-notification supervisor, maximum independent set.



Live demonstration, part A

Switch from the slides to the terminal and, side by side, the browser tab with the console (<http://127.0.0.1:7790>). Run one scenario at a time. Each one finishes in about 7 seconds and then holds; talk over the output, then press **Ctrl-C** before starting the next. Say this first: “Everything you are about to see is real: a mock AUR, the Warp log, three witnesses and three rebuilders, running as separate processes on this laptop. The ‘malicious’ packages are harmless probes that only read a planted decoy file and ping a local test server; nothing leaves the machine.”

Before the jury arrives (whoever owns the laptop)

```
cd loom
cargo build                # once; takes ~30 s
python3 demo/fixtures/genfix.py # materialise package fixtures
demo/run.sh                # full dry run, ~70 s; must end with 'demo complete'
cargo run -p loom-eval      # optional: prints 'ALL CHECKED ACCEPTANCE CRITERIA PAS'
```

- The first line of every run prints **sandbox tier on this host: full**. If it says *reduced*, the host has unprivileged user namespaces disabled; the demo still works, but say “reduced tier” when you describe the sandbox.
- Make the terminal font large (at least 16 pt) and the window at least 100 columns wide; the output is designed for 80 columns.
- Open a browser tab at <http://127.0.0.1:7790> (the live console; it is up only while a scenario is running with `LOOM_HOLD=1`).
- Open [docs/demo-report.html](#) in a second tab. It is a saved snapshot of a full run and is your backup if anything fails live.
- `demo/run.sh` frees its own ports, so a crashed earlier run is not a problem. Never run `pkill -f loomd` by hand.
- Each scenario is independent: `LOOM_HOLD=1 demo/run.sh <scenario>` runs one scenario in 6–10 seconds, then keeps the services and console alive until you press **Ctrl-C**.

Scenario 1 — a healthy package: verify, don't build

```
LOOM_HOLD=1 demo/run.sh healthy
```

What you will see

- Three rebuilders (thread-a, -b, -c) each print *reproducible* with the same short hash for libweft and hello-loom.
- **loom log**: tree size, the root hash, *cosigned by: witness-1, witness-2, witness-3*.
- **loom install hello-loom**: libweft is installed first (a dependency), both **ALLOWED**, *artifact ... from peer http://127.0.0.1:773x*, and *post_install() ran under Heddle (full; exit 0; 0 denial(s))*.

Say

“Three independent rebuilders reproduced these packages bit-for-bit and logged it. The log is cosigned by all three witnesses. Loom installed hello-loom by fetching the attested artifact from a peer by its hash, so **no PKGBUILD code ran on this machine**. Even the package's own install script ran inside the sandbox.”



Scenario 2 — temporal quarantine and the advisory fast-path

```
LOOM_HOLD=1 demo/run.sh quarantine
```

What you will see

- **fastmover 0.9-1 — BLOCKED (pre-build)**, [FAIL] temporal quarantine, *published 1h ago; quarantine is 3d — 2d 22h remaining*, and the exact override command.
- Then an advisory feed is loaded (CVE-2026-31337), and **tlsprobe 2.0.1-1 — ALLOWED**, although it is just as new.

Say

“fastmover was published an hour ago, so it is held for 72 hours even though all three rebuilders reproduced it. Worms like Shai-Hulud spread within hours and are usually caught within days, so a short delay removes most of their reach. tlsprobe is just as new, but the Arch security advisory feed says it fixes a vulnerability, so the quarantine is waived. We don't want to delay security fixes.”

Scenario 3 — the orphan-adoption attack

```
LOOM_HOLD=1 demo/run.sh orphan
```

What you will see

- orphan-tool 1.0 by **bob** installs normally.
- *published orphan-tool 1.1-1 by mallory*; then all three rebuilders print **buildfailed ... refused: build attempted openat /root/.curlrc [FR-3.2], socket ... AF_INET [FR-3.4]**.
- **orphan-tool 1.1-1 — BLOCKED (pre-build)**: [FAIL] publishing-authority continuity, *maintainer changed: bob → mallory*, with dates and the source of the evidence.

Say

“This models the 2026 AUR orphan-adoption waves. Bob abandoned the package; mallory adopted it and shipped 1.1. Loom remembers who published what you installed, notices the change of maintainer, and **blocks before building anything**.

Independently, look at the rebuilders: 1.1's build tried to read a file in root's home and open an internet socket. Heddle denied both, and the rebuilders refused to vouch for it. That is two separate mechanisms catching the same attack.”

Scenario 4 — build-time network injection (npm install inside a PKGBUILD)

```
LOOM_HOLD=1 demo/run.sh npm
```

What you will see

- npm-helper 2.4 installs. carol's account is compromised and 2.5 is published.
- All three rebuilders: **buildfailed ... refused: build attempted ...**
- **loom verify npm-helper --build → BLOCKED (pre-install)**, [FAIL] build confinement: *build tried to reach home/credential files and the network — contained by Heddle, artifact rejected (exit 0; 2 hostile access attempt(s))*, followed by the list of denials.
- **loom-testbed sink → (sink empty — no build reached the network)**.

Say

“This is the Atomic Arch pattern: a package whose build quietly runs npm install to fetch a payload. Notice **exit 0**: the script swallowed its own errors so the build looks successful. Loom does not care.



The build tried to use the network and read credentials, so it is hostile. And the test server, our sink, received nothing.”

Scenario 7 — the sandbox contrast: same build, confined vs unconfined

```
LOOM_HOLD=1 demo/run.sh sandbox
```

What you will see

- A decoy canary is planted in \$HOME.
- (a) confined: **BLOCKED** with the two denials.
- (b) UNCONFINED (demo backend, clearly marked): **ALLOWED WITH WARNINGS** — *DEMO BACKEND: build ran WITHOUT kernel confinement.*
- **network sink hits: confined=0 unconfined=1** and *HIT ... marker=loom-probe&pkg=npm-helper&stage=npm-install.*

Say

“This is our headline empirical result. The **same** malicious build, run twice. Confined by Heddle, zero hits: it could not read the home directory or reach the network. Unconfined, the way a normal AUR helper runs it, it phoned home. We don't just claim the sandbox works; we show it denying the payload.”

If it misbehaves: open the backup tab docs/demo-report.html and show the same result in the Heddle sandbox panel.

Handover line

“That was containment and quarantine. [Presenter 4] will now show the attacks that try to hide: a rewritten history with a planted hook, a Python startup hook, and an attack on the log itself.”
(Press Ctrl-C to stop the last scenario before handing over.)



Questions the jury is likely to ask you

Answer in two or three sentences, then stop. If a question belongs to another presenter's area, say "My teammate covered that, [name] can answer" and hand it over.

Q. Why not simply use Docker or bubblewrap?

Docker needs a root daemon and is built for isolation, not least-privilege policy; bubblewrap sets up namespaces but has no Landlock layer and does not explain denials. Heddle composes namespaces, Landlock and seccomp from a single declarative policy, so every denial names the requirement and rule that fired (FR-3.7), and it degrades to Landlock plus seccomp where user namespaces are disabled.

Q. What is Landlock, and why is it important here?

Landlock is a Linux security module (kernel 5.13+) that lets an unprivileged process restrict its own access to files; ABI 4 (kernel 6.7) adds TCP restrictions and ABI 6 (6.12) adds IPC scoping. It needs no root, so the sandbox still works in the reduced tier when namespaces are not allowed.

Q. What is seccomp-BPF?

A classic BPF program the kernel runs on every system call. It looks at the syscall number and arguments and returns allow, an error such as EPERM, or 'notify'. Loom returns EPERM for dangerous calls and, for socket and open calls, notifies a supervisor that logs and explains the denial. File access is always enforced by the namespace and Landlock, never by the supervisor alone, which avoids the known time-of-check-to-time-of-use race.

Q. Why does clone3 return ENOSYS?

seccomp can only inspect register arguments. clone3 passes its flags inside a struct in memory, so a filter cannot see whether it creates new namespaces. Returning 'not implemented' makes libc fall back to plain clone, whose flags are register arguments that the filter can check and refuse.

Q. Why block the x32 ABI?

x32 is an alternative system-call numbering on x86-64. If it were allowed, a program could reach a blocked system call through its x32 number and bypass the filter, so the filter rejects it (and kills on any foreign architecture).

Q. Won't the 'hostile denial' rule cause false positives?

Only two kinds of denial count as hostile: reading home or credential paths (FR-3.2) and network use (FR-3.4). Undeclared reads and restricted syscalls are only logged, because normal toolchains trigger them. A package that genuinely needs the network at build time gets a recorded, audited exception (FR-3.8). Our false-positive test (E2) blocks 0 of 7 benign packages.

Q. How is the number of independent rebuilders computed?

As a maximum independent set on a graph where two attestations are connected if they share a rebuilder, an organisation or a toolchain ID. Up to 20 attestations we search exactly with bitmasks; above that a greedy bound is used that can only under-count, never over-count.

Q. Why 72 hours of quarantine?

It is the SRS default and it is configurable. Fast worms are usually detected and pulled within days, and genuine security fixes skip the wait through the advisory fast-path, so the cost to users is small.

Numbers every presenter must know

Fact	Value	Where it comes from
Malicious cases blocked (AC-1)	9 / 9 (100%)	loom-eval E1, labelled incident corpus
Benign packages blocked (AC-2)	0 / 7 (0%)	loom-eval E2



Fact	Value	Where it comes from
Best single mechanism (AC-4)	catches 3 of 9 (sandbox)	loom-eval E4 ablation
Policy decision time	8.3 μ s (target < 500 ms)	loom-eval E5, release build
Ed25519 attestation verification	60 μ s (target < 100 ms)	loom-eval E5, release build
Split views detected (AC-6)	200 / 200	loom-eval E6
Sink hits, same malicious build	0 confined vs 1 unconfined	demo scenario 7
Code base	11 Rust crates, ~12,700 lines, 75 automated tests	cargo test --workspace
Default policy	k = 2 attestations, 2-of-3 witnesses, 72 h quarantine, continuity change = block	SRS Appendix A
Deployment in the demo	mock AUR :7700, Warp :7710, witnesses :7721-3, rebuilders/peers :7731-3, console :7790	demo/run.sh